```
Para Three
Selected paragraphs:
Para Two
```

The two selector expressions driving the example above demonstrate complex expressions composed of two combinators, child > with ~ subsequent sibling, and child > with + first sibling. You can build expressions as complex as you like, with any (or at least an arbitrarily large) number of simple/compound selectors.

I mentioned earlier that the `:has` pseudo-class is supported by PHP 8.4 and will probably prove the most useful and relevant. The main reason is that this is how CSS provides equivalent functionality to the XPath axes `ancestor::` and `parent::`. `*:has(p)` is equivalent to XPath `p/ancestor::*` and `*:has(> div)` is the same as `/div/parent::` … you get the idea. As always, here are some examples:

```
// ... $html5 heredoc as before
$dom = Dom\HTMLDocument::createFromString($html5);
$ancestor = $dom->querySelectorAll('*:has(p)');
print "Ancestors:" . PHP_EOL;
foreach ($ancestor as $node) {
    print $node->localName . PHP_EOL;
}
```

```
Output:
Ancestors:
html
body
main
```

If we changed the inner selector of `:has()` to use the child combinator > and feed it through the same `foreach` loop we get just the parent node:

```
// ... $html5 heredoc as before
$dom = Dom\HTMLDocument::createFromString($html5);
$parent = $dom->querySelectorAll('*:has(> p)');
print "Parent:" . PHP_EOL;
foreach ($parent as $node) {
    print $node->localName . PHP_EOL;
}
```